

EC340: Computer Organisation and Architecture

Pipelined processor Design

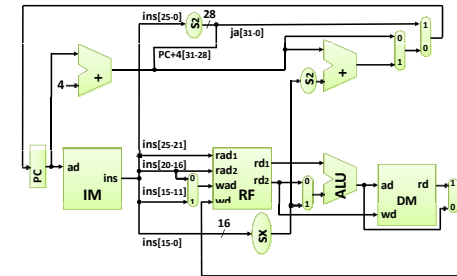
1

Outline

- Motivation for pipelining
- Clock period and CPI
- Pipelined data path
- Graphical representation
- Pipeline hazards

2

Single Cycle Datapath



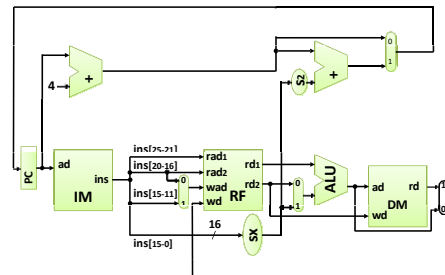
3

Problems with single cycle design

- Slowest instruction pulls down the clock frequency
- Resource utilization is poor
- There are some instructions which are impossible to be implemented in this manner

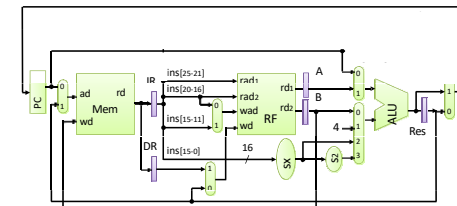
4

Single cycle datapath (excl "j")



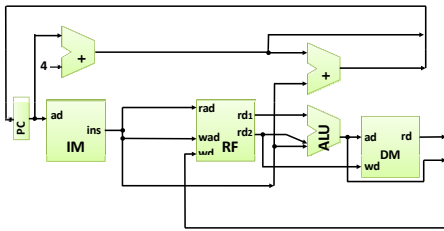
5

Multi-Cycle Data Path



6

Single cycle datapath (abstract)



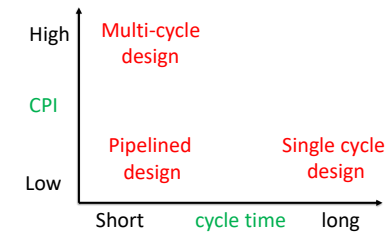
7

Resource utilization

- **Single cycle design**
 - Each resource is tied up for the entire duration of the instruction execution
- **Multi-cycle design**
 - Resource utilized in cycle t of the instruction I is available again for the cycle $t+1$ of the instruction I
- **Pipelined design**
 - Resource utilized in cycle t of instruction I is available again for cycle t of the instruction $I+1$

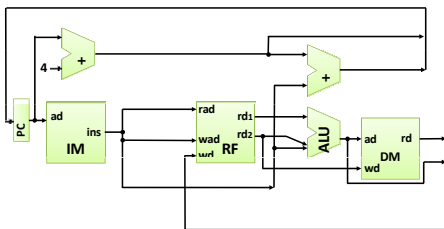
8

Cycle time and CPI



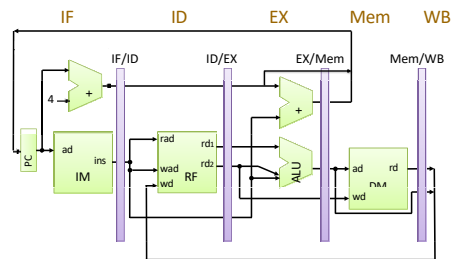
9

Single cycle datapath (abstract)



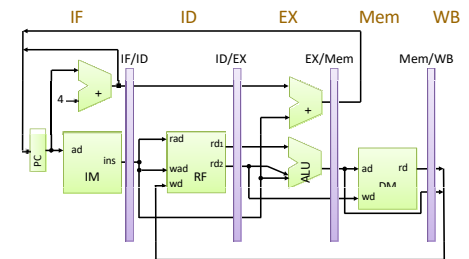
10

Pipelined datapath (abstract)



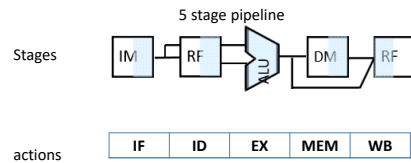
11

Fetch new instruction every cycle



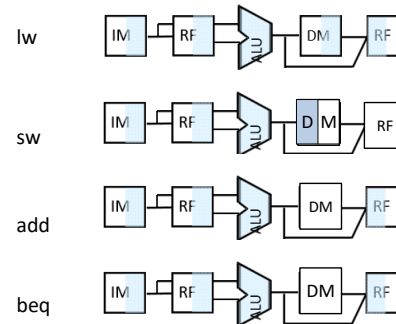
12

Graphical representation



13

Usage of stages by instructions



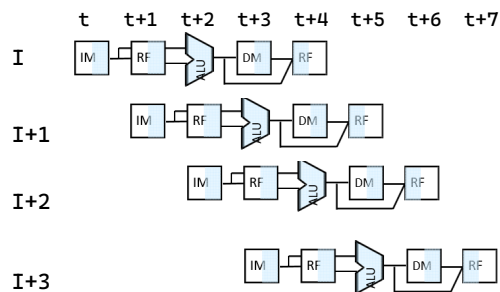
14

Representing pipelined execution

- Representation I
 - Horizontal axis: time
 - Vertical axis: instructions
- Representation II
 - Horizontal axis: pipeline stages
 - Vertical axis: time

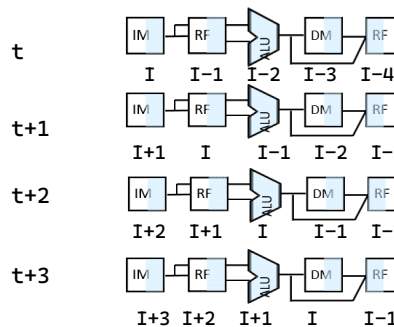
15

Representation I



16

Representation II



17

Hurdles in instruction pipelining

- **Structural hazards**
 - Resource conflicts – two instructions require same resource in the same cycle
- **Data hazards**
 - Data dependencies – one instruction needs data which is yet to be produced by another instruction
- **Control hazards**
 - Decision about next instruction needs more cycles

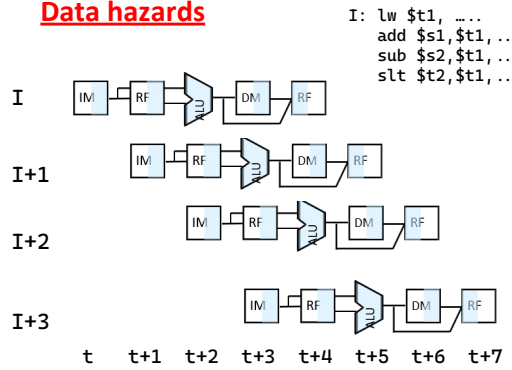
18

Structural hazards

- **No structural hazards in the present design**
 - Separate instruction and data memories
 - Adders for PC increment and offset addition to PC separate from main ALU
 - Each instruction uses ALU at most in one cycle
 - One instruction can read from RF while other can write into it in the same cycle

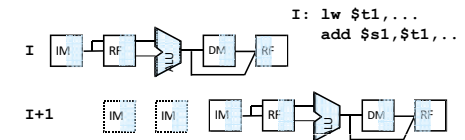
19

Data hazards



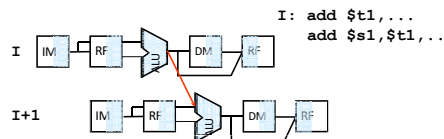
20

Stalls due to data hazards



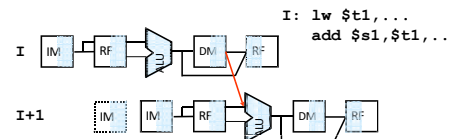
21

Data forwarding -1



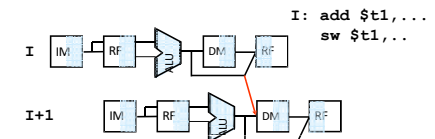
22

Data forwarding -2



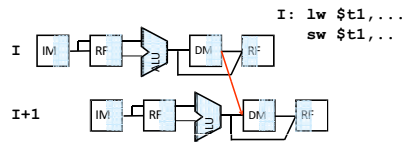
23

Data forwarding -3



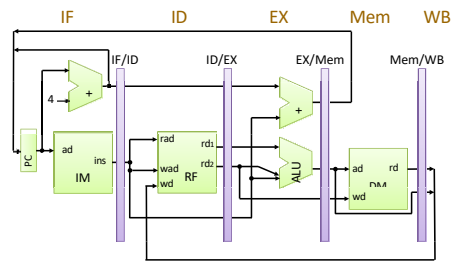
24

Data forwarding -4



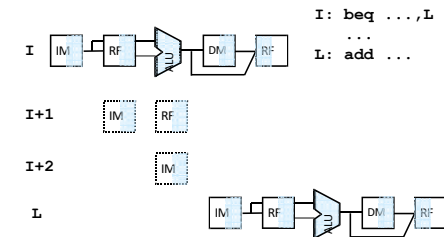
25

Fetch new instruction every cycle



26

Stalls due to control hazards



27

Summary

- Relationship between single cycle design, multi-cycle design, and pipelined design
- Fast clock, CPI = 1
- Pipelined data path: stages and interstage registers
- Representation: timewise or stagewise
- Structural hazards, data hazards, and control hazards

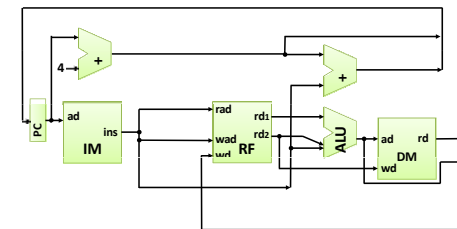
28

Outline

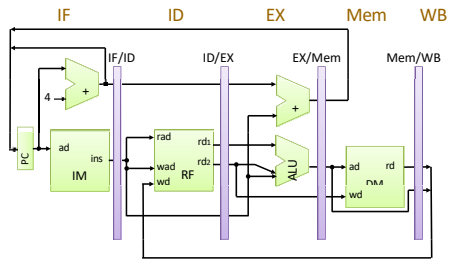
- Complete the data path design
- Design controller
- Considers the effect of hazards

29

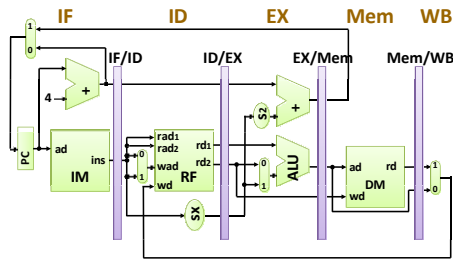
Single cycle datapath (abstract)



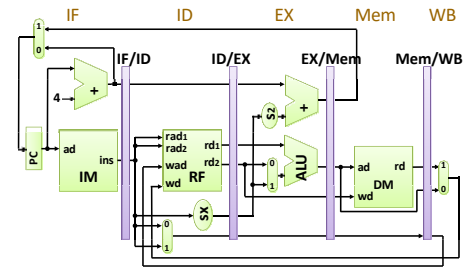
31

Pipelined datapath

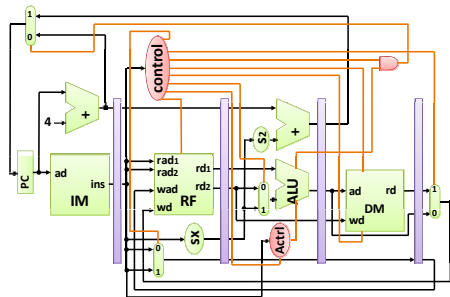
32

Put back multiplexers

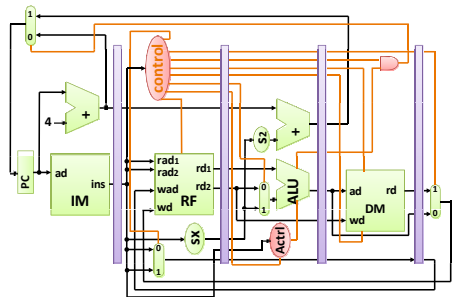
33

Correction for WB stage

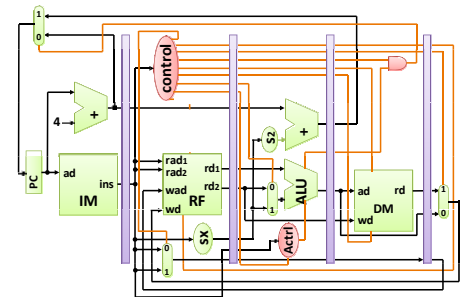
34

Abstract: Adding control

35

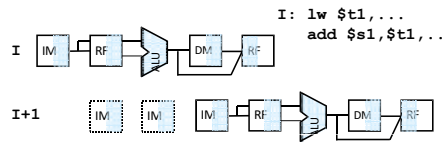
Control signals with delays

36

Correction for RF write signal

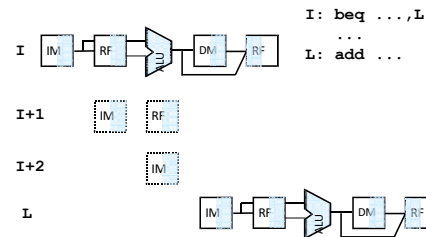
37

Stalls due to data hazards



38

Stalls due to control hazards



39

Handling hazards

- Data hazards
 - detect instructions with data dependence
 - introduce nop instructions (bubbles) in the pipeline
 - more complex: data forwarding
- Control hazards
 - detect branch instructions
 - flush inline instructions if branching occurs

40

Are there software solutions?

- Separate dependent instructions by reordering code
- Insert nop instruction in worst case
- Treat branches as delayed branches and insert suitable instructions in delay slots

41

Summary

- Datapath for pipelined processor = single cycle datapath plus interstage registers
- Controller same as single cycle controller with control signals passed through interstage registers
- WB stage requires special care
- Hazard handling requires detection, stalling, flushing
- Software solutions also exist

42

Outline

- Stalls due to data hazards
- Control to introduce stall cycles
- Detecting data hazard conditions
- Data forwarding paths
- Data forwarding control
- Stalls with data forwarding

43

Pipelined datapath

Pipelined datapath and control

Stalls due to data hazards

instruction view

I: lw \$t1,...
add \$s1,\$t1,...

Diagram illustrating stalls due to data hazards in an instruction view. The pipeline stages are IM (Instruction Memory), RF (Register File), ALU (Arithmetic Logic Unit), DM (Data Memory), and RF (Register File). The first instruction (I) is 'lw \$t1,...' and the second is 'add \$s1,\$t1,...'. The first instruction completes its execution. The second instruction stalls for two cycles because it needs the value of \$t1, which is not available until the first instruction's RF stage. The stalls are indicated by red 'X' and green checkmarks.

Red X: I+1 (stall)

Green checkmark: I+1 (stall)

Stalls due to data hazards

Stage-wise view

The diagram illustrates the stage-wise view of a 5-stage pipeline (IM, RF, ALU, DM, RF) over four time steps (t, t+1, t+2, t+3). It shows the progression of instructions I, I+1, and I+2. Instruction I+1 has a 'nop1' stall in the ALU stage at t+1. Instruction I+2 has 'nop2' stalls in the ALU and DM stages at t+2.

I: lw \$t1, ...
add \$s1, \$t1, ...

97

Stalls due to data hazards

instruction view again

The diagram illustrates the instruction view of a processor pipeline with stalls due to data hazards. The instructions are:

- I:** `lw $t1, ...` followed by `add $s1, $t1, ...`
- nop1**
- nop2**
- I+1**

The pipeline stages are IM (Instruction Memory), RF (Register File), ALU (Arithmetic Logic Unit), DM (Data Memory), and RF (Register File). The diagram shows that instruction I stalls for 2 cycles (nops) due to a data hazard with instruction I+1. The stalls are indicated by dashed boxes around the RF stage of instruction I and the IM and RF stages of instruction I+1.

Control for data hazard stalls

PCw=0
IF/Dw=0
bubble=1

49

Detecting data hazard

Condition to be checked:

Instruction in RF stage reads from a register in which instruction in ALU stage or DM stage is going to write

ID/EX.RW and

(IF/ID.rs=ID/EX.rd or IF/ID.rt=ID/EX.rd)

EX/DM.RW and

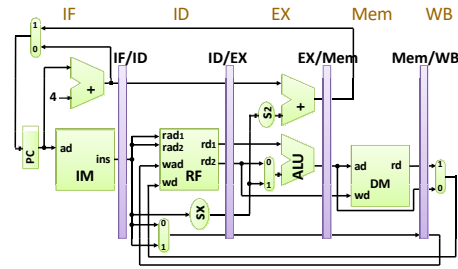
(IF/ID.rs=EX/DM.rd or IF/ID.rt=EX/DM.rd)

We need to ensure that instruction in RF actually reads rs and/or rt (not taken care here)

Note: rd is the RF destination address after multiplexing

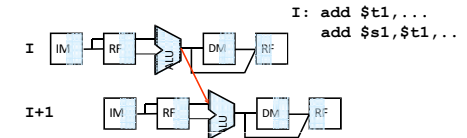
50

Pipelined datapath



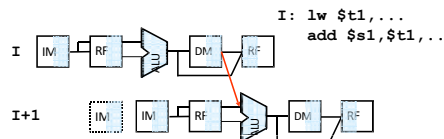
51

Data forwarding path P1



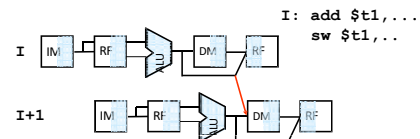
52

Data forwarding path P2



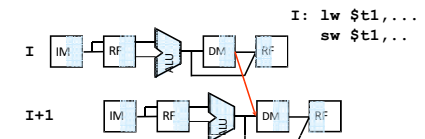
53

Data forwarding path P3



54

Data forwarding path P4



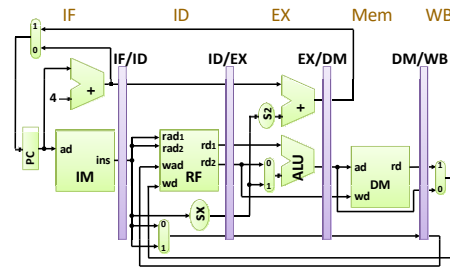
55

Data forwarding path list

- P1
from ALU out (EX/DM) to ALU in1/2
- P2
from DM/ALU out (DM/WB) to ALU in1/2
- P3/P4
from DM/ALU out (DM/WB) to DM in

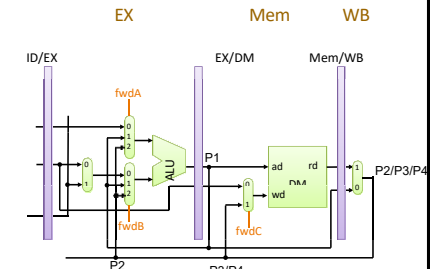
56

Pipelined datapath



57

Actual forwarding paths



58

Data forwarding control - P1, P2

P1: from ALU out (EX/DM) to ALU in1/2
 P2: from DM/ALU out (DM/WB) to ALU in1/2

fwdA=1 if EX/DM.RW and EX/DM.rd≠0 and
 EX/DM.rd = ID/EX.rs
 fwdA=2 if DM/WB.RW and DM/WB.rd≠0 and
 EX/DM.rd ≠ ID/EX.rs
 DM/WB.rd = ID/EX.rs

For fwdB, replace rs with rt

59

Data forwarding control - P3/P4

P3/P4: from DM/ALU out (DM/WB) to DM in

fwdC=1 if DM/WB.RW and DM/WB.rd≠0 and
 DM/WB.rd = EX/DM.rt

60

Stalling with data forwarding

Condition to be checked:

Instruction in RF stage reads from a register in which a **1w** instruction in ALU stage is going to write

ID/EX.MR and

(IF/ID.rs=ID/EX.rd or IF/ID.rt=ID/EX.rd)

We need to ensure that instruction in RF actually reads rs and/or rt (not taken care here)

Note: rd is the RF destination address after multiplexing

61

Summary

- Data hazards and stalls seen in different views
- Control signals for stalling instructions and inserting nops
- Checking hazard conditions
- Three types of forwarding paths
- Control for data forwarding
- Inserting stalls with data forwarding

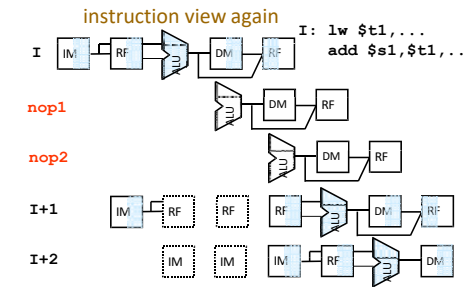
62

Outline

- Modifying design to handle control hazards
- Improve branch performance – branch elimination, branch speed up, branch prediction

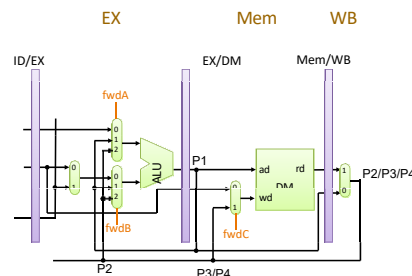
63

Stalls due to data hazards



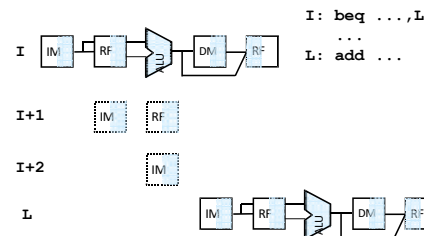
64

Actual forwarding paths



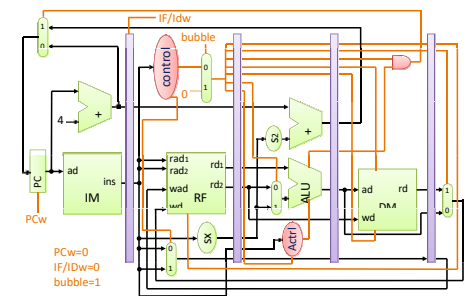
65

Stalls due to control hazards



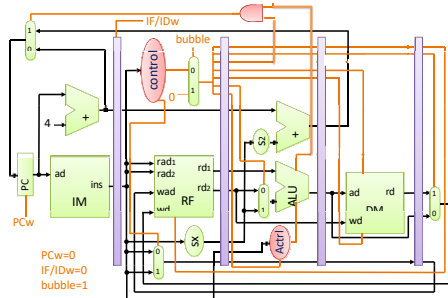
66

Current design: branch in 4th cycle



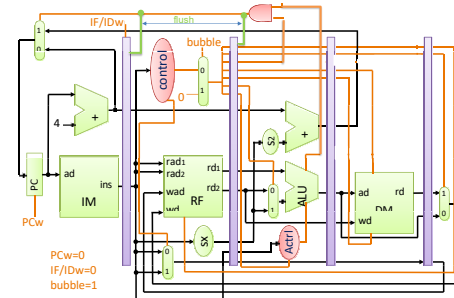
67

Current design: branch in 3rd cycle



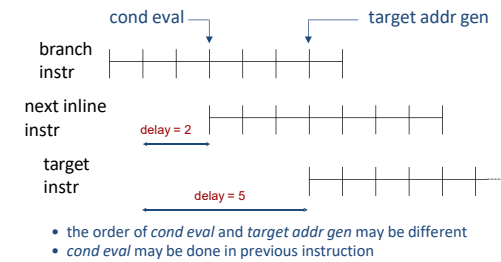
68

Flushing instructions in RF and ALU



69

Control Hazards



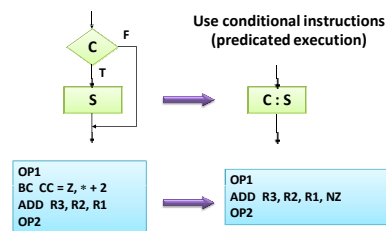
70

Improving Branch Performance

- Branch Elimination
 - replace branch with other instructions
- Branch Speed Up
 - reduce time for computing evaluation and target instruction fetch
- Branch Prediction
 - guess the outcome and proceed, undo if necessary
- Branch Target Capture
 - make use of history

71

Branch Elimination



72

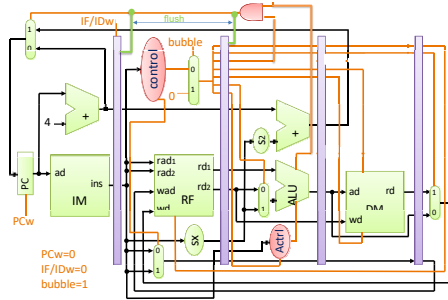
Branch Speed Up :

early target address/condition evaluation

- Assume each instruction is Branch
- Generate target address while decoding
- After decoding discard target address if not Branch
- Evaluate condition as early as possible

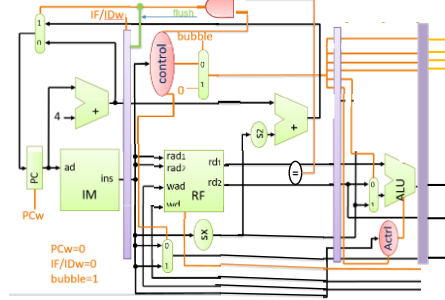
73

Flushing instructions in RF and ALU



74

Design with branch speed up



75

Branch Prediction

- Treat conditional branches as unconditional branches/ NOP
- Undo if necessary

Strategies

- Fixed (always guess inline)
- Static (guess on the basis of instruction type and/or target address)
- Dynamic (guess based on recent history)

77

Dynamic Branch Prediction - basic idea

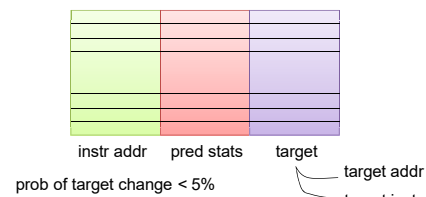
Predict based on the history of previous branch

loop: xxx 2 miss-predictions
xxx for every
xxx occurrence
xxx
BC loop

78

Branch Target Capture

- Branch Target Buffer (BTB)
- Target Instruction Buffer (TIB)



79

Correlation between branches

B1: if (x) • B3 can be predicted
... with 100% accuracy
B2: if (y) based on the
... outcomes of B1 and
z = x && y B2
B3: if (z)
...

80

Pipelining and exception handling

- Multiple exception possible in a cycle
- Order of exceptions may not be the same as the order of instruction – precise and imprecise interrupts
- Status saving is complex
- Partially executed instructions may be flushed and re-executed on return

81

Summary

- Stalls due to control hazards
- Flushing incorrect instructions
- Improving branch performance
 - Branch elimination
 - Branch speed up
 - Branch prediction – static/dynamic

82